



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Sistema Inteligente de Gestión Logística con ArangoDB

Realizado por

**Ismael Carrasco Mkhazni
Guillermo García León**

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Este proyecto presenta el diseño e implementación de una plataforma de gestión logística avanzada utilizando una base de datos multimodelo (ArangoDB). El sistema permite gestionar una red de almacenes y productos mediante grafos, optimizando el cálculo de rutas y el control de inventarios en tiempo real. La solución se despliega mediante una arquitectura contenedorizada con Docker y una interfaz reactiva en Streamlit.

Palabras clave: ArangoDB, Grafos, Logística, AQL, Docker, Streamlit.

Abstract

This project presents the design and implementation of an advanced logistics management platform using a multi-model database (ArangoDB). The system allows managing a network of warehouses and products using graphs, optimizing route calculation and real-time inventory control. The solution is deployed using a containerized architecture with Docker and a reactive interface in Streamlit.

Keywords: ArangoDB, Graphs, Logistics, AQL, Docker, Streamlit.

Índice general

1	Ejemplos de uso de LaTeX	1
1.1.	Introducción	1
1.2.	Estilos	1
1.3.	Listados	1
1.4.	Subsecciones	2
1.4.1.	Primera subsección	2
1.4.2.	Segunda subsección	2
1.5.	Imágenes y figuras	2
1.6.	Tablas	3
1.7.	Referencias	4
1.8.	Extractos de código	4
1.9.	Enlaces	6
1.10.	Citas y bibliografía	6
1.11.	Ecuaciones	6
1.12.	Caracteres y símbolos especiales	7
2	Introducción	8
3	Estudio previo	9
3.1.	Introducción	9
3.2.	ArangoDB	9
3.3.	Arquitectura Multimodelo	9
3.4.	Modelado de Grafos y AQL	9
3.5.	Integración	9
4	Análisis del problema	10
4.1.	Introducción	10
4.2.	Requisitos de información	10
4.3.	Requisitos funcionales	10
4.4.	Conclusiones	10
5	Diseño de la solución	12
5.1.	Introducción	12
5.2.	Modelo de Datos y Diagrama	12
5.3.	Decisiones de Diseño: Integridad en Grafos	13
5.4.	Conclusiones	13
6	Implementación	14
6.1.	Introducción	14
6.2.	Herramientas y Tecnologías	14
6.3.	Arquitectura: Patrón Repositorio	14

6.4. Resolución de Conflictos Técnicos	14
6.5. Conclusiones	15
7 Pruebas	16
7.1. Introducción	16
7.2. Validación del Algoritmo de Rutas	16
7.3. Simulación de Contingencias	16
7.4. Conclusiones	16
8 Conclusiones	17
Bibliografía	18

Índice de figuras

1.1. Un feroz depredador	2
5.1. Diagrama topológico del grafo RedLogística y sus relaciones.	12

Índice de tablas

1.1. Tabla LaTeX de ejemplo	3
1.2. Tabla compleja introducida como figura	3

Índice de extractos de código

1.1. Código Python	4
1.2. Código Java	5
1.3. Código HTML	5
1.4. Código JavaScript	5

1. Ejemplos de uso de LaTeX

Este capítulo se incluye únicamente como ayuda y referencia de uso de LaTeX. No debe aparecer en el documento final.

1.1. Introducción

En este capítulo se muestran ejemplos de uso de LaTeX para operaciones comunes.

1.2. Estilos

Se pueden aplicar estilos al texto como **negritas**, *cursiva*, subrayado y monoespaciado. También se **pueden** aplicar colores, y combinar estilos. Se recomienda usar sólo negritas para hacer énfasis, y no abusar de este recurso.

Por comodidad para usuarios no habituados con LaTeX, esta plantilla define algunos alias de comandos más fáciles de recordar para estilos de texto comunes: **negritas**, *cursiva* y código.

1.3. Listados

Con itemize se pueden crear listas no numeradas:

- Fresas
- Melocotones
- Piñas
- Nectarinas

De manera similar, enumerate permite crear listas numeradas:

1. Elaborar la memoria del TFG
2. Elaborar la presentación
3. Presentar el TFG
4. Solicitar el título de Grado

1.4. Subsecciones

Se pueden definir subsecciones con el comando subsection:

1.4.1. Primera subsección

Esto es una subsección

1.4.2. Segunda subsección

Esto es otra subsección.

Sub-sub-sección

Este es un tercer nivel de profundidad, que no aparece en el índice general. Se recomienda no utilizarlo, si es posible.

1.5. Imágenes y figuras

Todas las imágenes y figuras del documento se incluirán en la carpeta “figures”. Se pueden incluir de la siguiente manera:



Figura 1.1: Un feroz depredador

Observe que las figuras se numeran automáticamente según el capítulo y el número de figuras que hayan aparecido anteriormente en dicho capítulo. Existen muchas maneras de definir el tamaño de una figura, pero se aconseja utilizar la mostrada en este ejemplo: se define el ancho de la figura como un porcentaje del

ancho total de la página, y la altura se escala automáticamente. De esta manera, el ancho máximo de una figura sería $1.0 * \text{textwidth}$, lo que aseguraría que se muestra al máximo tamaño posible sin sobrepasar los márgenes del documento.

Tenga en cuenta que LaTeX intenta incluir las figuras en el mismo sitio donde se declaran, pero en ocasiones no es posible por motivos de espacio. En esos casos, LaTeX colocará la figura lo más cerca posible de su declaración, puede que en una página diferente. Esto es un comportamiento normal.

1.6. Tablas

Existe una gran variedad de formas de crear tablas en LaTeX puro, y todas ellas tienen cierta complejidad. A continuación se muestra un ejemplo simple de tabla nativa, en la Tabla 1.1. Se recomienda crear un archivo en la carpeta *tables* por cada tabla nativa que se desee incluir, y enlazarla mediante el comando `input`.

Columna L	Columna C	Columna R	Columna P
Texto de ejemplo	Texto de ejemplo	Texto de ejemplo	Texto de ejemplo
ABC	DEF	HIJ	KLM

Tabla 1.1: Tabla LaTeX de ejemplo

Para tablas con un formato más complejo, considere la posibilidad de diseñarla usando otro software externo (por ejemplo Excel) e incluirla de manera similar a una figura. **Observe en el código LaTeX a continuación cómo usar el comando `captionof{table}`, en lugar de simplemente `caption`, hace que se liste como una Tabla en lugar de como una Figura:**

		%			Comparisons		
Group	N	positive	neutral	negative	χ^2	df	p
Q1 (a)	Programming Labs are useful for learning the subject						
CS group	400	73	7	20	0.9	4	ns
Math group	400	70	8	22			
Q1 (b)	Working with data makes predictions more reliable						
CS group	400	52	12	36	13.7	4	< 0.05
Math group	400	44	15	41			
Q1 (c)	Lessons in class are more active than web conferences						
CS group	400	46	11	43	14.8	4	< 0.01
Math group	400	31	14	55			

Tabla 1.2: Tabla compleja introducida como figura

1.7. Referencias

Observe cómo en el código fuente de esta sección se ha usado varias veces el comando `label`. Este comando permite marcar un elemento, ya sea capítulo, sección, figura, etc. para hacer una referencia numérica al mismo. Para referenciar una label se usa el comando `ref` incluyendo el nombre de la referencia:

Este es el capítulo [1](#).

En la sección [1.2](#) se muestran ejemplos de estilos.

La subsección [1.4.1](#) explica...

En la Figura [1.1](#) vemos que...

Esto evita que tengamos que escribir directamente los índices de las secciones y figuras que queremos mencionar, ya que LaTeX lo hace por nosotros y además se encarga de mantenerlos actualizados en caso de que cambien (pruebe a mover este capítulo al final del documento y observe cómo se actualizan automáticamente todos los índices referenciados). Además, las referencias mediante “`ref`” actúan como hipervínculos dentro del documento que llevan al elemento referenciado al pulsar en ellas.

Es habitual nombrar las “label” con un prefijo que indica el tipo de elemento para encontrarlo luego más fácilmente, pero no es obligatorio.

1.8. Extractos de código

Se pueden incluir extractos de código mediante `lstlisting`:

```
1 num = float(input("Enter a number: "))
2 if num > 0:
3     print("Positive number")
4 elif num == 0:
5     print("Zero")
6 else:
7     print("Negative number")
```

Extracto de código 1.1: Código Python

Para evitar tener que incluir el código directamente en el texto del documento, se pueden guardar en archivos separados y referenciarlos:

Los extractos de código también se pueden referenciar mediante `label/ref`: Extractos de código [1.1](#), [1.2](#), [1.3](#), [1.4](#).

```
1 class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, LaTeX!");  
4     }  
5 }
```

Extracto de código 1.2: Código Java

```
1 <html>  
2     <head>  
3         <title>LaTeX example</title>  
4     </head>  
5  
6     <body>  
7         Hello, LaTeX!  
8     </body>  
9 </html>
```

Extracto de código 1.3: Código HTML

```
1 function main() {  
2     console.log("Hello, LaTeX!");  
3 }
```

Extracto de código 1.4: Código JavaScript

1.9. Enlaces

Puede enlazar una web externa mediante el comando `url`: <https://www.example.com>. También se puede vincular un enlace a un texto mediante el comando `href`: [dominio de ejemplo](#).

1.10. Citas y bibliografía

En LaTeX, los elementos de la bibliografía se almacenan en un fichero bibliográfico en un formato llamado BibTeX, en el caso de este proyecto se encuentran en “bibliografia.bib”. Para citar un elemento se usa el comando `cite`. Se pueden citar tanto artículos científicos [1] como enlaces web [2].

También se puede usar el comando `citet` para incluir una referencia junto con el nombre de su autor o autores: Borrego et al. [3]. Todas las citas se numeran automáticamente y se incluyen en la sección de bibliografía del trabajo. El orden por defecto es según su orden de aparición en el documento. Para ordenarlas por orden alfabético del autor, puede modificar el comando `bibliographystyle` del archivo principal y reemplazar su valor por el estilo `plainnat` (orden alfabético, nombres completos) o `abbrvnat` (orden alfabético, nombres abreviados).

Observe cómo los elementos bibliográficos almacenados en “bibliografia.bib” tienen una etiqueta asociada, que es la que se usa al citarlos mediante `cite`. **Añadir una referencia al fichero bibliográfico no hace que ésta aparezca automáticamente en la sección de bibliografía del trabajo, es necesario citarla en algún lugar del mismo.**

1.11. Ecuaciones

LaTeX tiene un potente motor para mostrar ecuaciones matemáticas y un amplio catálogo de símbolos matemáticos. El entorno matemático se puede activar de muchas maneras. Para incluir ecuaciones simples en un texto se pueden rodear de símbolos dólar: $1 + 2 = 3$, $\sqrt{81} = 3^2 = 9$, $\forall x \in y \exists z : S_z < 4$.

Las ecuaciones más complejas pueden expresarse aparte y son numeradas: ecuación 1.1.

$$\lim_{x \rightarrow 0} \frac{e^x - 1}{2x} \stackrel{\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}}{H} \lim_{x \rightarrow 0} \frac{e^x}{2} = \frac{1}{2} + 7 \int_0^2 \left(-\frac{1}{4} (e^{-4t_1} + e^{4t_1-8}) \right) dt_1 \quad (1.1)$$

Dispone [aquí](#) de un amplio listado de símbolos que pueden usarse en modo matemático.

1.12. Caracteres y símbolos especiales

Algunos caracteres y símbolos deben ser escapados para poder representarse en el documento, ya que tienen un significado especial en LaTeX. Algunos de ellos son:

- El símbolo dólar \$ se usa para ecuaciones.
- El tanto por ciento % se usa para comentarios en el código fuente.
- El símbolo euro € suele dar problemas si se escribe directamente.
- El guión bajo _ se usa para subíndices en modo matemático.
- Las comillas deben expresarse ‘así’ para comillas simples y “así” para comillas dobles. Las comillas españolas pueden expresarse «así».
- La barra invertida o contrabarra \ se usa para comandos LaTeX.
- Otros símbolos que deben escaparse son las llaves { }, el ampersand &, la almohadilla # y los símbolos mayor que > y menor que <.

2. Introducción

El propósito de este trabajo es resolver la complejidad inherente a la logística de distribución mediante el uso de bases de datos de grafos. En lugar de utilizar modelos relacionales tradicionales que sufren con consultas recursivas pesadas al intentar modelar redes, he optado por ArangoDB para modelar almacenes como nodos y rutas como aristas, permitiendo una adaptabilidad inmediata ante cambios en el entorno.

La metodología seguida ha sido iterativa: partiendo de un diseño de esquema básico, progresé hasta lograr una arquitectura modular robusta, enfrentándome a retos técnicos tanto en la lógica de resolución de rutas óptimas mediante AQL como en la orquestación del despliegue en contenedores Docker, los cuales se detallan en los posteriores capítulos de implementación.

3. Estudio previo

3.1. Introducción

Breve introducción al capítulo

3.2. ArangoDB

Es una base de datos NoSQL nativa multimodelo, lo que significa que permite trabajar con documentos, grafos y sistemas clave-valor utilizando un único motor central y un mismo lenguaje de consultas llamado AQL (ArangoDB Query Language).

3.3. Arquitectura Multimodelo

En el mundo de las bases de datos, normalmente estás atado a un solo formato: tablas (relacional), documentos tipo JSON (como MongoDB), o grafos (como Neo4j).

La principal ventaja de la arquitectura multimodelo nativa de ArangoDB es que combina varios modelos en un mismo motor central, permitiendo acceder a ellos con el mismo lenguaje de consultas (AQL). Los tres modelos que soporta son:

- **Documentos:** Para guardar información jerárquica y flexible.
- **Grafos:** Para definir relaciones complejas y direcciones entre los datos.
- **Clave-Valor:** Para accesos de lectura ultrarrápidos mediante un identificador único.

3.4. Modelado de Grafos y AQL

En una base de datos tradicional, buscar conexiones requiere cruzar tablas mediante operaciones costosas. En un modelo de grafos, las relaciones son importantes y trazarlas es directo e inmediato. AQL (ArangoDB Query Language) es el lenguaje que permite recorrer esta red de datos. .

3.5. Integración

4. Análisis del problema

4.1. Introducción

Este capítulo detalla los requisitos fundamentales que el sistema logístico debe satisfacer para considerarse operativo y resolver el problema propuesto.

4.2. Requisitos de información

El sistema requiere persistir y relacionar las siguientes entidades fundamentales:

- **Almacenes (Nodos):** Identificación geográfica y capacidad total.
- **Productos (Nodos):** Catálogo de artículos disponibles en la red.
- **Conexiones Viales (Aristas):** Rutas físicas entre almacenes, cuantificadas por su distancia en kilómetros.
- **Existencias (Aristas):** Cantidad disponible de un producto específico almacenado en una ubicación concreta.

4.3. Requisitos funcionales

Las principales funcionalidades exigidas al sistema son:

- **Gestión de Red (CRUD):** Permitir el alta, baja y modificación de almacenes, productos y rutas viales.
- **Cálculo de Rutas Óptimas:** Determinar el camino más corto entre dos ubicaciones basándose en la métrica de distancia.
- **Control de Stock Vivo:** Gestionar inventarios con operaciones de entrada y salida actualizadas en tiempo real.
- **Resiliencia Logística:** Capacidad de recalcular rutas de transporte automáticamente ante contingencias (ej. cierre temporal de vías).

4.4. Conclusiones

La fuerte interconexión de estos requisitos apunta de forma evidente hacia la adopción de un modelo de datos orientado a grafos, donde las relaciones (las rutas

y el stock) poseen tanta importancia técnica y valor de negocio como las propias entidades aisladas.

5. Diseño de la solución

5.1. Introducción

La base de datos se ha estructurado como un grafo multimodelo denominado RedLogística. A diferencia de las bases de datos relacionales estándar, este modelo me permite recorrer conexiones logísticas con una eficiencia de orden de complejidad mucho menor.

5.2. Modelo de Datos y Diagrama

La organización lógica de las colecciones diseñadas en ArangoDB es la siguiente:

- **Vértices (Documentos):** Almacenes y Productos.
- **Aristas (Edges):** Rutas (conecta almacenes entre sí, incluyendo el atributo `distancia_km`) y Stock_en (conecta productos con almacenes, incluyendo el atributo de cantidad).

En la Figura 5.1 se ilustra la topología de la base de datos mediante un diagrama conceptual.

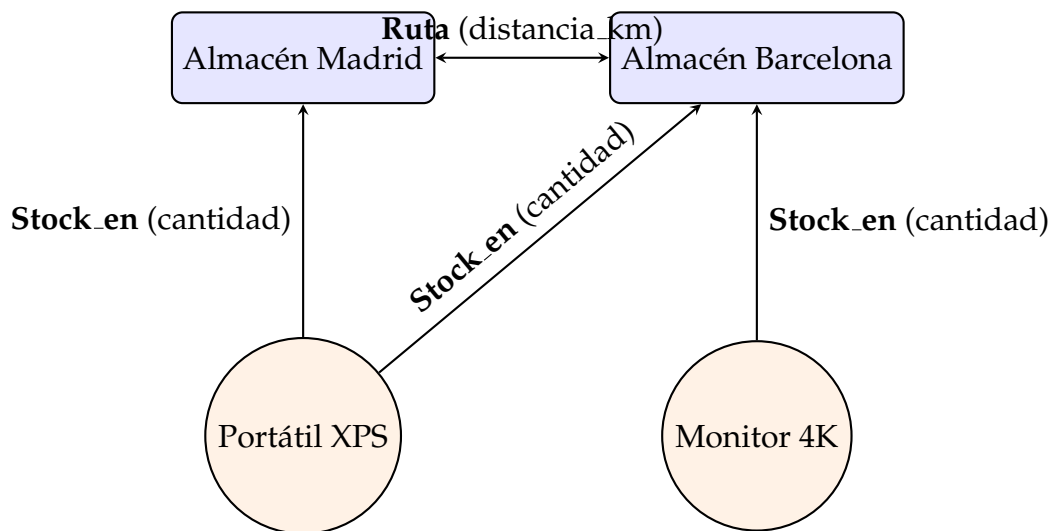


Figura 5.1: Diagrama topológico del grafo RedLogística y sus relaciones.

5.3. Decisiones de Diseño: Integridad en Grafos

Una decisión arquitectónica crítica a la que tuve que enfrentarme fue la gestión de la integridad referencial. Dado que ArangoDB no impone restricciones rígidas de borrado en cascada de la misma forma que un motor SQL, tuve que implementar lógica AQL manual transaccional. Esto garantiza que, si se elimina un almacén del sistema por necesidades de negocio, se supriman simultáneamente todas sus rutas adyacentes y registros de stock vinculados, previniendo la existencia de "aristas huérfanas" que pudieran corromper consultas futuras.

5.4. Conclusiones

El diseño propuesto explota las ventajas de los grafos nativos para resolver un problema de red complejo, garantizando al mismo tiempo la consistencia de los datos mediante una capa lógica de control implementada directamente en el código de backend.

6. Implementación

6.1. Introducción

Este capítulo describe el proceso de construcción real del sistema, detallando el stack tecnológico seleccionado y las soluciones de ingeniería aplicadas para organizar el código y sortear los problemas de despliegue.

6.2. Herramientas y Tecnologías

- **Base de Datos:** ArangoDB configurado para persistencia de grafos multimodelo.
- **Backend:** Python 3.10 utilizando el controlador oficial `python-arango`.
- **Frontend:** Streamlit, empleado para desarrollar una interfaz reactiva.
- **Despliegue:** Orquestación de servicios mediante `docker-compose`, aislando la base de datos (db) de la aplicación (app).

6.3. Arquitectura: Patrón Repositorio

A medida que el proyecto crecía, la clase gestora original corría el riesgo de convertirse en un antipatrón *God Class* (una única clase abarcando toda la funcionalidad). Para refactorizarlo, apliqué el Patrón Repositorio dividiendo el sistema en módulos lógicos:

1. `database.py`: Dedicado en exclusiva a la gestión de la conexión y la instanciación.
2. `crud.py`: Contiene clases aisladas (`CRUDAlmacenes`, `CRUDRutas`, etc.) que abstraen la lógica AQL de cada dominio.

6.4. Resolución de Conflictos Técnicos

Durante la integración de Docker, me enfrenté a un error de importación circular en Python (`ModuleNotFoundError`) provocado por una colisión de *namespaces* al renombrar el directorio principal de la aplicación. Logré solucionarlo modificando el punto de entrada a `main.py` y, fundamentalmente, configurando la variable de entorno `PYTHONPATH=/app` dentro del contenedor, forzando al intérprete a resolver correctamente la jerarquía de directorios.

6.5. Conclusiones

La implementación resultante demuestra ser modular, mantenible y altamente portable gracias al esfuerzo puesto en la correcta contenedorización del entorno de ejecución.

7. Pruebas

7.1. Introducción

Las pruebas del sistema se han enfocado en validar la exactitud algorítmica en el recorrido de grafos y en comprobar la capacidad dinámica del sistema frente a alteraciones inesperadas en la topología de la red.

7.2. Validación del Algoritmo de Rutas

Durante la evaluación empírica de las consultas AQL, detecté una anomalía semántica importante: el algoritmo `SHORTEST_PATH` nativo tendía a utilizar los nodos de Productos como .atajos.^{entre} ciudades, dado que estas aristas no contaban con atributo de distancia. Esto devolvía rutas carentes de sentido físico (ej. 350 km entre Madrid y Valencia saltando a través de un producto).

Logré resolver este comportamiento restringiendo explícitamente el motor AQL para que evaluara de manera exclusiva la colección de aristas Rutas, obligando al algoritmo de Dijkstra interno a considerar únicamente carreteras reales. Tras aplicar esta restricción, las métricas fueron exactas.

7.3. Simulación de Contingencias

Para verificar la resiliencia del modelo, desarrollé un test de estrés eliminando deliberadamente la arista principal que conectaba Madrid y Barcelona (simulando un accidente o corte vial). Al volver a solicitar el recálculo de la ruta óptima, el sistema reaccionó con una adaptabilidad inmediata: redirigió el flujo logístico a través de nodos intermedios alternativos (vía Bilbao) y ajustó la distancia total de 620 km a 1010 km de manera totalmente transparente, sin requerir una sola modificación en la lógica de backend.

7.4. Conclusiones

Las pruebas confirman el éxito rotundo del modelo orientado a grafos. El sistema reacciona en tiempo real a los cambios estructurales, demostrando que ArangoDB es una solución técnica superior para la planificación logística.

8. Conclusiones

El desarrollo integral de este sistema logístico ha servido para validar de forma empírica la idoneidad de las bases de datos de grafos para la resolución de problemas de enrutamiento que, históricamente, penalizan severamente el rendimiento en sistemas de naturaleza relacional o tabular.

El uso de un diseño multimodelo en ArangoDB ha simplificado drásticamente el modelo mental del dominio de negocio. Paralelamente, la adopción del Patrón Repositorio en la capa de software ha asegurado un código legible y escalable. La contenedorización completa del ciclo de vida del proyecto mediante Docker me ha permitido ofrecer una solución "llave en mano", mitigando la clásica problemática de las dependencias locales y garantizando un despliegue transparente para su evaluación.

Aunque la curva de aprendizaje inicial del lenguaje AQL y del razonamiento transversal en grafos fue pronunciada, los resultados técnicos conseguidos demuestran una capacidad resolutive sólida y justifican plenamente la elección tecnológica, logrando una plataforma altamente tolerante a fallos logísticos.

Uso de Herramientas de IA

En cumplimiento con las directrices de la asignatura "Complementos de Bases de Datos", declaro el uso de modelos generativos de IA (Gemini, de Google) como herramienta de apoyo técnico durante el desarrollo del proyecto. Sus usos específicos han sido:

- **Análisis Arquitectónico y Refactorización:** Soporte consultivo en la transición del código de un modelo centralizado a uno distribuido (Patrón Repositorio) para la separación de responsabilidades en la base de datos.
- **Depuración de Despliegue:** Asistencia técnica puntual en la comprensión de jerarquías de directorios dentro de Docker para resolver el fallo de importación circular (PYTHONPATH).
- **Formato y Estilo:** Apoyo en la adaptación y formateo de la documentación técnica al lenguaje de marcado LaTeX.

Como autor de la entrega, declaro que todo el código fuente y el contenido teórico generado ha sido analizado de manera crítica, depurado y validado empíricamente a través de mis pruebas. Comprendo en profundidad cada bloque lógico y componente del sistema expuesto en esta memoria.

Bibliografía

- [1] Agustín Borrego, Daniel Ayala, Inma Hernández, Carlos R. Rivero, and David Ruiz. Generating rules to filter candidate triples for their correctness checking by knowledge graph completion techniques. In *K-CAP*, pages 115–122, 2019. doi: 10.1145/3360901.3364418.
- [2] Universidad de Sevilla. Página principal de la escuela técnica de ingeniería informática, 2020. URL <https://www.informatica.us.es>.
- [3] Agustín Borrego, Daniel Ayala, Inma Hernández, Carlos R. Rivero, and David Ruiz. CAFE: Knowledge graph completion using neighborhood-aware features. *Eng. Appl. Artif. Intell.*, 103:104302, 2021. doi: 10.1016/j.engappai.2021.104302. URL <https://doi.org/10.1016/j.engappai.2021.104302>.